

# Django REST Framework

Complete Guide — Scratch to Production

10 Stages · 60+ Topics · Theory + Code + Production Patterns Generated May 2026

| What you'll learn           |                                  |
|-----------------------------|----------------------------------|
| Python & Django Foundations | REST API Design Principles       |
| Serializers & Validation    | Authentication & Permissions     |
| ViewSets & Routers          | Filtering, Search & Pagination   |
| Performance & Caching       | Testing & Documentation          |
| Async, Tasks & WebSockets   | Security & Production Deployment |

---

# Table of Contents

---

## Stage 1 — Python & Django Foundations

- › Python OOP for Django
- › Django MTV Pattern
- › Django ORM
- › URL Routing
- › Django Settings
- › manage.py Commands

## Stage 2 — DRF Core Concepts

- › Installing DRF
- › Serializers
- › ModelSerializer
- › APIView
- › `@api_view`
- › Renderers & Parsers
- › Validation

## Stage 3 — Views, Routers & ViewSets

- › GenericAPIView & Mixins
- › ViewSets
- › DefaultRouter & SimpleRouter
- › HyperlinkedSerializer
- › `@action` Decorator
- › Nested Routes

## Stage 4 — Authentication & Permissions

- › SessionAuthentication
- › TokenAuthentication
- › JWT with SimpleJWT
- › OAuth2
- › Built-in Permission Classes
- › Custom Permissions

## Stage 5 — Filtering, Search & Pagination

- › SearchFilter
- › OrderingFilter
- › django-filter
- › PageNumberPagination
- › CursorPagination
- › LimitOffsetPagination

## Stage 6 — Performance & Caching

- › `select_related`
- › `prefetch_related`
- › Django Cache Framework

- 
- › django-cachalot
  - › Response Caching
  - › Database Indexing

## **Stage 7 — Testing & Documentation**

- › APIClient
- › pytest-django
- › factory\_boy
- › drf-spectacular
- › Swagger UI & Redoc
- › Coverage.py

## **Stage 8 — Async, Tasks & Real-time**

- › Celery + Redis
- › Celery Beat
- › Django Channels & WebSockets
- › Async Views
- › Webhooks
- › Server-Sent Events

## **Stage 9 — Security Hardening**

- › Throttling
- › CORS
- › HTTPS Enforcement
- › Sensitive Data Masking
- › Input Sanitisation
- › django-axes

## **Stage 10 — Deployment & Production Ops**

- › Gunicorn / Uvicorn
- › Docker & Docker Compose
- › Nginx Reverse Proxy
- › Cloud Deployment
- › Sentry & Logging
- › CI/CD Pipelines

# Stage 1 · Python & Django Foundations

Everything you must know before writing a single line of DRF code

## 1.1 Python OOP for Django Developers

Django and DRF are heavily class-based. Understanding Python's object-oriented model — classes, inheritance, mixins, decorators, and dunder methods — is non-negotiable. DRF's serializers, views, and permissions are all class hierarchies that you will subclass daily.

### Classes, Inheritance & `super()`

Python supports single and multiple inheritance. DRF uses multiple inheritance (mixins) extensively. Always call `super().__init__()` to preserve the MRO chain.

#### ■ Python OOP fundamentals used in DRF

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof"

# Mixin pattern – used heavily inside DRF
class TimestampMixin:
    def created_at(self):
        import datetime
        return datetime.datetime.utcnow()

class AuditedDog(TimestampMixin, Dog):
    pass

d = AuditedDog("Rex")
print(d.speak()) # Rex says Woof
print(d.created_at()) # 2025-...
```

## Decorators

Decorators wrap a function or method to extend its behaviour. DRF's `@api_view`, `@permission_classes`, `@authentication_classes`, and `@action` are all decorators.

#### ■ Writing and using decorators

```
import functools, time

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.perf_counter()-start:.4f}s")
        return result
    return wrapper

@timer
def slow_query():
    time.sleep(0.1)
    return "done"

slow_query() # slow_query took 0.1001s
```

## Properties & Class Methods

#### ■ Properties and classmethods

```
class User:
    def __init__(self, first, last):
        self.first = first
        self.last = last

    @property
    def full_name(self): # accessed as user.full_name
        return f"{self.first} {self.last}"

    @classmethod
    def from_dict(cls, data): # alternative constructor
        return cls(data['first'], data['last'])

u = User.from_dict({'first': 'Jane', 'last': 'Doe'})
print(u.full_name) # Jane Doe
```

■ DRF's `Serializer.to_representation()` and `Serializer.to_internal_value()` are prime examples of the *Template Method pattern* — override them in subclasses to customise behaviour without rewriting the whole pipeline.

## 1.2 Django MTV Pattern

---

Django follows Model-Template-View (MTV), which maps to MVC as: Model=Model, Template=View, View=Controller. DRF replaces the Template layer with Serializers and the View layer with APIViews.

## Request Lifecycle

- Browser/client sends HTTP request
- Django's WSGI/ASGI handler receives it
- Middleware stack processes it (auth, session, CORS, ...)
- URL dispatcher matches the path to a view
- View calls the model, gets data, passes to serializer
- Response is returned through the middleware stack

### ■ Minimal Django app structure

```
myproject/  
manage.py  
myproject/  
settings.py # CONFIG: INSTALLED_APPS, DATABASES, ...  
urls.py # ROOT URL CONF  
wsgi.py / asgi.py  
myapp/  
models.py # Database schema  
serializers.py # DRF: data in/out  
views.py # DRF: business logic  
urls.py # App-level URL conf  
admin.py  
apps.py  
tests.py
```

## 1.3 Django ORM

The ORM lets you interact with the database using Python objects instead of raw SQL. Every DRF serializer and viewset ultimately talks to the ORM. Understanding QuerySets, managers, and relations is essential for performance.

## Models & Migrations

### ■ models.py — a realistic example

```
from django.db import models  
from django.contrib.auth.models import AbstractUser  
  
class User(AbstractUser):  
    bio = models.TextField(blank=True)  
    avatar = models.ImageField(upload_to='avatars/', null=True, blank=True)
```

```

class Meta:
    db_table = 'users'
    indexes = [models.Index(fields=['email'])]

class Category(models.Model):
    name = models.CharField(max_length=100, unique=True)
    slug = models.SlugField(unique=True)

    def __str__(self):
        return self.name

class Article(models.Model):
    STATUS_DRAFT = 'draft'
    STATUS_PUBLISHED = 'published'
    STATUS_CHOICES = [(STATUS_DRAFT, 'Draft'), (STATUS_PUBLISHED, 'Published')]

    title = models.CharField(max_length=255, db_index=True)
    slug = models.SlugField(unique=True)
    body = models.TextField()
    author = models.ForeignKey(User, on_delete=models.CASCADE,
                               related_name='articles')
    categories = models.ManyToManyField(Category, blank=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES,
                              default=STATUS_DRAFT, db_index=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-created_at']
        indexes = [models.Index(fields=['status', 'created_at'])]

    def __str__(self):
        return self.title

```

## ■ QuerySets — CRUD and filtering

```

# CREATE
article = Article.objects.create(
    title="Hello DRF", slug="hello-drf",
    body="Content here", author=user, status='published'
)

# READ — lazy evaluation
qs = Article.objects.filter(status='published').select_related('author')
qs = qs.prefetch_related('categories')
qs = qs.order_by('-created_at')[:10] # LIMIT 10

# Single object (raises DoesNotExist if missing)

```

```

article = Article.objects.get(slug="hello-drf")

# UPDATE
Article.objects.filter(author=user).update(status='draft')

# DELETE
article.delete()

# AGGREGATION
from django.db.models import Count, Avg
stats = Article.objects.aggregate(
    total=Count('id'),
    avg_len=Avg('body__len') # requires Func or annotation
)

```

■ *Never call `list()` or iterate a `QuerySet` inside a loop without prefetching related objects first. That's the N+1 query problem — covered in Stage 6.*

## 1.4 URL Routing

Django's URL dispatcher maps URL patterns to view functions or classes. DRF Routers automate this for ViewSets (Stage 3), but understanding manual routing is necessary for custom endpoints.

### ■ `myproject/urls.py`

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/v1/', include('articles.urls')), # versioned
    path('api/v1/auth/', include('auth.urls')),
]

```

### ■ `articles/urls.py`

```

from django.urls import path
from . import views

app_name = 'articles'

urlpatterns = [
    path('articles/', views.ArticleListCreateView.as_view(), name='list-create'),
    path('articles/<int:pk>/', views.ArticleDetailView.as_view(), name='detail'),
    path('articles/<slug:slug>/', views.ArticleBySlugView.as_view(), name='by-slug'),
]

```



## 1.5 Django Settings (Production-Ready)

Never keep a single settings.py for all environments. Use a settings package with base, development, and production modules.

### ■ settings/base.py

```
from pathlib import Path
import os, environ

BASE_DIR = Path(__file__).resolve().parent.parent

env = environ.Env()
environ.Env.read_env(BASE_DIR / '.env')

SECRET_KEY = env('SECRET_KEY')
DEBUG = env.bool('DEBUG', default=False)
ALLOWED_HOSTS = env.list('ALLOWED_HOSTS', default=[])

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Third-party
    'rest_framework',
    'rest_framework.authtoken',
    'corsheaders',
    'django_filters',
    'drf_spectacular',
    # Local
    'articles',
    'users',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware', # must be first
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

```

DATABASES = {
    'default': env.db('DATABASE_URL',
        default='sqlite:///db.sqlite3')
}

AUTH_USER_MODEL = 'users.User'

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    'DEFAULT_PAGINATION_CLASS':
        'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20,
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
}

```

## 1.6 manage.py Commands

### ■ Essential manage.py commands

```

# Development server
python manage.py runserver 0.0.0.0:8000

# Database migrations
python manage.py makemigrations # detect model changes
python manage.py migrate # apply migrations
python manage.py migrate myapp 0003 # migrate to specific version
python manage.py showmigrations # list all

# Django shell (interactive)
python manage.py shell
python manage.py shell_plus # django-extensions: auto-imports

# Create superuser
python manage.py createsuperuser

# Static files (production)

```

---

```
python manage.py collectstatic --no-input

# Check for deployment issues
python manage.py check --deploy

# Custom management command skeleton
# myapp/management/commands/seed_data.py
from django.core.management.base import BaseCommand

class Command(BaseCommand):
    help = 'Seed initial data'

    def handle(self, *args, **options):
        self.stdout.write('Seeding...')

        # your logic

        self.stdout.write(self.style.SUCCESS('Done'))
```

# Stage 2 · DRF Core Concepts

Serializers, Views, Renderers — the beating heart of every DRF API

## 2.1 Installing DRF

### ■ Install and configure

```
pip install djangorestframework
pip install markdown # for browsable API
pip install django-filter # filtering support

# settings.py
INSTALLED_APPS = [
    ...
    'rest_framework',
]
```

## 2.2 Serializers — Deep Dive

A Serializer converts complex Python objects (model instances, querysets) into native Python datatypes that can be rendered into JSON/XML, and vice versa. It also handles validation of incoming data before saving.

### How serialization works internally

- `serializer.data` → calls `to_representation()` → `OrderedDict`
- `serializer.validated_data` → runs `run_validators()` + `field.validate()` + `validate_()` + `validate()`
- `serializer.save()` → calls `create()` or `update()` depending on instance presence

### ■ serializers.py — base Serializer

```
from rest_framework import serializers
from .models import Article, Category

class CategorySerializer(serializers.Serializer):
    id = serializers.IntegerField(read_only=True)
    name = serializers.CharField(max_length=100)
    slug = serializers.SlugField()

    def create(self, validated_data):
        return Category.objects.create(**validated_data)
```

```
def update(self, instance, validated_data):
    instance.name = validated_data.get('name', instance.name)
    instance.slug = validated_data.get('slug', instance.slug)
    instance.save()
    return instance

# Usage
data = {'name': 'Django', 'slug': 'django'}
ser = CategorySerializer(data=data)
if ser.is_valid(raise_exception=True):
    category = ser.save()

# Serialise an existing object
ser = CategorySerializer(category)
print(ser.data) # OrderedDict([('id', 1), ('name', 'Django'), ...])
```

## 2.3 ModelSerializer

ModelSerializer is a Serializer subclass that auto-generates fields from a model. It also provides default create() and update() implementations.

### ■ ModelSerializer — full featured

```
class ArticleSerializer(serializers.ModelSerializer):
    author_name = serializers.SerializerMethodField()
    word_count = serializers.SerializerMethodField()
    categories = CategorySerializer(many=True, read_only=True)
    category_ids = serializers.PrimaryKeyRelatedField(
        queryset=Category.objects.all(),
        many=True, write_only=True, source='categories')
    url = serializers.HyperlinkedIdentityField(
        view_name='articles:detail', lookup_field='slug')

    class Meta:
        model = Article
        fields = ['id', 'url', 'title', 'slug', 'body', 'status',
                  'author_name', 'word_count', 'categories', 'category_ids',
                  'created_at', 'updated_at']
        read_only_fields = ['id', 'slug', 'created_at', 'updated_at']
        extra_kwargs = {
            'body': {'write_only': True}, # only in write requests
        }

    def get_author_name(self, obj):
        return obj.author.get_full_name()
```

```

def get_word_count(self, obj):
    return len(obj.body.split())

def validate_title(self, value):
    if len(value) < 5:
        raise serializers.ValidationError("Title must be at least 5 chars.")
    return value.strip()

def validate(self, attrs):
    # Object-level validation
    if attrs.get('status') == 'published' and not attrs.get('body'):
        raise serializers.ValidationError("Published articles must have a body.")
    return attrs

def create(self, validated_data):
    categories = validated_data.pop('categories', [])
    request = self.context['request']
    article = Article.objects.create(
        author=request.user, **validated_data)
    article.categories.set(categories)
    return article

def update(self, instance, validated_data):
    categories = validated_data.pop('categories', None)
    for attr, val in validated_data.items():
        setattr(instance, attr, val)
    instance.save()
    if categories is not None:
        instance.categories.set(categories)
    return instance

```

■ *Always pass context={‘request’: request} when instantiating serializers inside views — HyperlinkedFields and SerializerMethodFields often need the request to build absolute URLs.*

## 2.4 APIView

APIView is the DRF counterpart of Django's View. It wraps the request in a DRF Request object, handles content negotiation, and dispatches to get/post/put/patch/delete methods.

### ■ views.py — APIView

```

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from django.shortcuts import get_object_or_404

```

```

from .models import Article
from .serializers import ArticleSerializer

class ArticleListCreateView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        articles = Article.objects.filter(status='published') .select_related('author')
        .prefetch_related('categories')
        serializer = ArticleSerializer(
            articles, many=True, context={'request': request})
        return Response(serializer.data)

    def post(self, request):
        serializer = ArticleSerializer(
            data=request.data, context={'request': request})
        serializer.is_valid(raise_exception=True) # auto 400 on failure
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)

class ArticleDetailView(APIView):
    def get_object(self, slug):
        return get_object_or_404(Article, slug=slug)

    def get(self, request, slug):
        article = self.get_object(slug)
        serializer = ArticleSerializer(article, context={'request': request})
        return Response(serializer.data)

    def put(self, request, slug):
        article = self.get_object(slug)
        serializer = ArticleSerializer(
            article, data=request.data, context={'request': request})
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data)

    def patch(self, request, slug):
        article = self.get_object(slug)
        serializer = ArticleSerializer(
            article, data=request.data,
            partial=True, context={'request': request})
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data)

    def delete(self, request, slug):
        article = self.get_object(slug)

```

```
article.delete()
return Response(status=status.HTTP_204_NO_CONTENT)
```

## 2.5 @api\_view — Function-Based Views

For simple, one-off endpoints, `@api_view` is faster to write than `APIView`. It converts a plain function into a DRF view that handles content negotiation.

### ■ Function-based DRF views

```
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated

@api_view(['GET', 'POST'])
@permission_classes([IsAuthenticated])
def article_list(request):
    if request.method == 'GET':
        articles = Article.objects.filter(status='published')
        serializer = ArticleSerializer(articles, many=True,
            context={'request': request})
        return Response(serializer.data)

    serializer = ArticleSerializer(data=request.data,
        context={'request': request})
    serializer.is_valid(raise_exception=True)
    serializer.save(author=request.user)
    return Response(serializer.data, status=201)
```

## 2.6 Renderers & Parsers

Renderers control how the response is formatted (JSON, XML, HTML browsable API). Parsers control how incoming request bodies are decoded (JSON, form data, multipart).

### ■ settings.py — custom renderers & parsers

```
REST_FRAMEWORK = {
    'DEFAULT_RENDERER_CLASSES': [
        'rest_framework.renderers.JSONRenderer',
        'rest_framework.renderers.BrowsableAPIRenderer', # dev only
    ],
    'DEFAULT_PARSER_CLASSES': [
        'rest_framework.parsers.JSONParser',
        'rest_framework.parsers.MultiPartParser', # file uploads
        'rest_framework.parsers.FormParser',
```



```
],  
}
```

#### ■ Per-view renderer override

```
from rest_framework.renderers import JSONRenderer  
  
class ArticleListCreateView(APIView):  
    renderer_classes = [JSONRenderer] # only JSON, no browsable  
    ...
```

## 2.7 Validation — All Levels

DRF validates data at three levels: field-level, object-level, and via validators. Validation errors automatically return HTTP 400 with a structured error response.

#### ■ All validation levels

```
from rest_framework import serializers  
from rest_framework.validators import UniqueValidator, UniqueTogetherValidator  
  
class ArticleSerializer(serializers.ModelSerializer):  
    title = serializers.CharField(  
        validators=[  
            UniqueValidator(  
                queryset=Article.objects.all(),  
                message="An article with this title already exists."  
            )  
        ]  
    )  
  
    class Meta:  
        model = Article  
        fields = '__all__'  
        validators = [  
            UniqueTogetherValidator(  
                queryset=Article.objects.all(),  
                fields=['author', 'slug']  
            )  
        ]  
  
        # Field-level validation  
        def validate_title(self, value):  
            if value.lower().startswith('test'):  
                raise serializers.ValidationError("Don't name articles 'test...'" )  
            return value
```

---

```
# Object-level validation
def validate(self, data):
    if data['status'] == 'published' and len(data.get('body','')) < 100:
        raise serializers.ValidationError({
            'body': 'Published articles need at least 100 characters.'
        })
    return data
```

# Stage 3 · Views, Routers & ViewSets

Cut boilerplate by 80% with Generic Views, ViewSets, and Routers

## 3.1 GenericAPIView & Mixins

GenericAPIView extends APIView with built-in queryset, serializer\_class, pagination, and filter support. Combine it with mixins to get list/create/retrieve/update/destroy actions without writing boilerplate.

■ views.py — GenericAPIView + Mixins

```
from rest_framework import generics, mixins, permissions

class ArticleListCreateView(
    mixins.ListModelMixin,
    mixins.CreateModelMixin,
    generics.GenericAPIView
):
    queryset = Article.objects.published().select_related('author')
    serializer_class = ArticleSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]

    def get(self, request, *args, **kwargs):
        return self.list(request, *args, **kwargs)

    def post(self, request, *args, **kwargs):
        return self.create(request, *args, **kwargs)

# --- DRF provides convenient shortcuts: ---
class ArticleListCreateView(generics.ListCreateAPIView):
    queryset = Article.objects.all().select_related('author')
    serializer_class = ArticleSerializer

class ArticleDetailView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Article.objects.all()
    serializer_class = ArticleSerializer
    lookup_field = 'slug' # use slug instead of pk

    def get_queryset(self):
        # Per-request queryset customisation
        user = self.request.user
        if user.is_staff:
            return Article.objects.all()
```

```

return Article.objects.filter(status='published')

def perform_update(self, serializer):
    # Hook called inside update() – great for audit logs
    serializer.save(updated_by=self.request.user)

```

■ *Use `get_queryset()` instead of `queryset=` whenever the `queryset` depends on the request (user, URL params). A class-level `queryset` is evaluated once at startup.*

## 3.2 ViewSets

A `ViewSet` groups related views into a single class. `ModelViewSet` provides list, create, retrieve, update, `partial_update`, and `destroy` out of the box. Routers then wire all of these to URLs automatically.

### ■ `views.py` — `ModelViewSet`

```

from rest_framework import viewsets, permissions, filters
from django_filters.rest_framework import DjangoFilterBackend

class ArticleViewSet(viewsets.ModelViewSet):
    queryset = Article.objects.all().select_related('author')
    .prefetch_related('categories')
    serializer_class = ArticleSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]
    filter_backends = [DjangoFilterBackend, filters.SearchFilter,
filters.OrderingFilter]
    filterset_fields = ['status', 'author']
    search_fields = ['title', 'body']
    ordering_fields = ['created_at', 'title']
    ordering = ['-created_at']
    lookup_field = 'slug'

    def get_permissions(self):
        if self.action in ['list', 'retrieve']:
            return [permissions.AllowAny()]
        return [permissions.IsAuthenticated()]

    def get_serializer_class(self):
        if self.action == 'list':
            return ArticleListSerializer # lighter serializer for lists
        return ArticleDetailSerializer

    def perform_create(self, serializer):
        serializer.save(author=self.request.user)

```

---

## 3.3 Routers

---

### ■ urls.py — Router wiring

```
from rest_framework.routers import DefaultRouter
from .views import ArticleViewSet, CategoryViewSet

router = DefaultRouter()
router.register('articles', ArticleViewSet, basename='article')
router.register('categories', CategoryViewSet, basename='category')

# DefaultRouter generates:
# GET /articles/ -> list
# POST /articles/ -> create
# GET /articles/{slug}/ -> retrieve
# PUT /articles/{slug}/ -> update
# PATCH /articles/{slug}/ -> partial_update
# DELETE /articles/{slug}/ -> destroy

urlpatterns = router.urls
```

## 3.4 @action Decorator

---

Use `@action` to add custom endpoints to a `ViewSet` beyond the standard CRUD. For example: `/articles/{slug}/publish/`, `/users/me/`

### ■ Custom actions inside ViewSet

```
from rest_framework.decorators import action
from rest_framework.response import Response

class ArticleViewSet(viewsets.ModelViewSet):
    ...

    @action(detail=True, methods=['post'],
            permission_classes=[IsAuthenticated])
    def publish(self, request, slug=None):
        article = self.get_object()
        if article.author != request.user:
            return Response({'error': 'Not yours'}, status=403)
        article.status = 'published'
        article.save()
        return Response({'status': 'published'})

    @action(detail=False, methods=['get'])
    def trending(self, request):
        # /api/v1/articles/trending/
```

---

```
qs = Article.objects.annotate(
    view_count=Count('views')
).order_by('-view_count')[:10]
ser = self.get_serializer(qs, many=True)
return Response(ser.data)

@action(detail=False, methods=['get'],
    permission_classes=[IsAuthenticated])
def my_articles(self, request):
    qs = Article.objects.filter(author=request.user)
    ser = self.get_serializer(qs, many=True)
    return Response(ser.data)
```

# Stage 4 · Authentication & Permissions

Who are you? What are you allowed to do?

## 4.1 SessionAuthentication

SessionAuthentication uses Django's session framework and CSRF tokens. Best suited for browser-based clients (SPA with same-domain cookies). DRF enforces CSRF automatically when using sessions.

### ■ settings.py + login view

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.SessionAuthentication',
    ],
}

# Login via Django's built-in view
# urls.py
from django.contrib.auth import views as auth_views
urlpatterns += [
    path('auth/login/', auth_views.LoginView.as_view(), name='login'),
    path('auth/logout/', auth_views.LogoutView.as_view(), name='logout'),
]
```

## 4.2 TokenAuthentication

Token auth generates a per-user opaque token stored in the database. The client sends Authorization: Token header. Simple but tokens don't expire — add `knox` or rotate manually for production.

### ■ Token auth setup & usage

```
# settings.py
INSTALLED_APPS += ['rest_framework.authtoken']

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
    ],
}
```

```
# Generate tokens for all existing users (once)
from django.db import migrations

def create_tokens(apps, schema_editor):
    from rest_framework.authtoken.models import Token
    User = apps.get_model('auth', 'User')
    for user in User.objects.all():
        Token.objects.get_or_create(user=user)

# views.py — login endpoint
from rest_framework.authtoken.views import obtain_auth_token
urlpatterns += [path('auth/token/', obtain_auth_token)]

# Client usage:
# POST /auth/token/ {"username":"jane","password":"secret"}
# -> {"token": "abc123..."}
# Subsequent requests:
# Authorization: Token abc123...
```

## 4.3 JWT Authentication (SimpleJWT)

JSON Web Tokens are stateless — no database lookup needed to validate. SimpleJWT provides access tokens (short-lived, 5 min) and refresh tokens (long-lived, 7 days). This is the recommended approach for production APIs.

### ■ JWT setup

```
pip install django-rest-framework-simplejwt

# settings.py
from datetime import timedelta

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=15),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True, # new refresh on each use
    'BLACKLIST_AFTER_ROTATION': True,
    'ALGORITHM': 'HS256',
    'AUTH_HEADER_TYPES': ('Bearer',),
    'USER_ID_FIELD': 'id',
    'USER_ID_CLAIM': 'user_id',
```



```

}

# urls.py
from rest_framework_simplejwt.views import (
    TokenObtainPairView, TokenRefreshView, TokenVerifyView
)
urlpatterns += [
    path('auth/token/', TokenObtainPairView.as_view()),
    path('auth/token/refresh/', TokenRefreshView.as_view()),
    path('auth/token/verify/', TokenVerifyView.as_view()),
]

```

#### ■ Custom JWT claims

```

from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from rest_framework_simplejwt.views import TokenObtainPairView

class MyTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        # Add custom claims
        token['email'] = user.email
        token['username'] = user.username
        token['is_staff'] = user.is_staff
        return token

class MyTokenObtainPairView(TokenObtainPairView):
    serializer_class = MyTokenObtainPairSerializer

```

## 4.4 Custom Permission Classes

Implement `has_permission()` for view-level checks and `has_object_permission()` for object-level checks.

#### ■ permissions.py

```

from rest_framework.permissions import BasePermission, SAFE_METHODS

class IsOwnerOrReadOnly(BasePermission):
    """
    Object-level: only the owner can write.
    Anyone can read published articles.
    """
    message = "You must be the owner of this object."

    def has_permission(self, request, view):

```

```
# View-level: allow reads, require auth for writes
if request.method in SAFE_METHODS:
    return True
return request.user and request.user.is_authenticated

def has_object_permission(self, request, view, obj):
    if request.method in SAFE_METHODS:
        return True
    return obj.author == request.user

class IsStaffOrReadOnly(BasePermission):
    def has_permission(self, request, view):
        if request.method in SAFE_METHODS:
            return True
        return request.user.is_staff

class IsEmailVerified(BasePermission):
    message = "Please verify your email address first."

    def has_permission(self, request, view):
        return (request.user.is_authenticated and
                request.user.profile.email_verified)

# Apply in view
class ArticleViewSet(viewsets.ModelViewSet):
    permission_classes = [IsOwnerOrReadOnly, IsEmailVerified]
```

# Stage 5 · Filtering, Search & Pagination

Make your list endpoints fast, flexible, and client-friendly

## 5.1 SearchFilter

### ■ Search across multiple fields

```
from rest_framework import filters

class ArticleViewSet(viewsets.ModelViewSet):
    filter_backends = [filters.SearchFilter]
    # ^ prefix: case-insensitive contains
    # = prefix: exact match
    # @ prefix: full-text search (MySQL only)
    # $ prefix: regex
    search_fields = ['title', 'body', 'author__username',
                    '=status', 'categories__name']

    # Client: GET /api/articles/?search=django
```

## 5.2 OrderingFilter

### ■ Ordering

```
class ArticleViewSet(viewsets.ModelViewSet):
    filter_backends = [filters.OrderingFilter]
    ordering_fields = ['created_at', 'title', 'author__username']
    ordering = ['-created_at'] # default

    # Client: GET /api/articles/?ordering=title,-created_at
```

## 5.3 django-filter

django-filter provides powerful declarative filtering with support for range, date, choice, and relationship filters.

### ■ filters.py — FilterSet

```
import django_filters
from .models import Article
```

```

class ArticleFilter(django_filters.FilterSet):
    status = django_filters.ChoiceFilter(choices=Article.STATUS_CHOICES)
    author = django_filters.CharFilter(field_name='author__username',
        lookup_expr='iexact')
    created_after = django_filters.DateFilter(field_name='created_at',
        lookup_expr='gte')
    created_before = django_filters.DateFilter(field_name='created_at',
        lookup_expr='lte')
    category = django_filters.CharFilter(field_name='categories__slug')

    class Meta:
        model = Article
        fields = ['status', 'author', 'category']

# views.py
from django_filters.rest_framework import DjangoFilterBackend

class ArticleViewSet(viewsets.ModelViewSet):
    filter_backends = [DjangoFilterBackend]
    filterset_class = ArticleFilter

# Client: GET /api/articles/?status=published&created_after=2024-01-01

```

## 5.4 Pagination

### ■ pagination.py — all three types

```

from rest_framework.pagination import (
    PageNumberPagination, LimitOffsetPagination, CursorPagination
)

# 1. Page-number pagination (GET /articles/?page=2&page_size=10)
class StandardPagination(PageNumberPagination):
    page_size = 20
    page_size_query_param = 'page_size'
    max_page_size = 100
    page_query_param = 'page'

# 2. Limit-offset (GET /articles/?limit=10&offset=20)
class FlexiblePagination(LimitOffsetPagination):
    default_limit = 20
    max_limit = 200

# 3. Cursor (opaque, stable for infinite scroll)
class CursorPagination(CursorPagination):
    page_size = 20
    ordering = '-created_at' # must be a unique, stable field

```

---

```
cursor_query_param = 'cursor'

# Apply globally
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'myapp.pagination.StandardPagination',
    'PAGE_SIZE': 20,
}

# Or per-viewset
class ArticleViewSet(viewsets.ModelViewSet):
    pagination_class = CursorPagination
```

# Stage 6 · Performance & Caching

Make your API fast enough that nobody notices the network

## 6.1 Eliminating N+1 Queries

The N+1 problem: fetching 100 articles triggers 100 extra queries to fetch each author. `select_related` does a SQL JOIN for ForeignKey/OneToOne. `prefetch_related` does a separate query and joins in Python for ManyToMany/reverse FK.

### ■ Optimised queryset

```
# BAD: 1 query for articles + N queries for each author + M for categories
Article.objects.all()

# GOOD
Article.objects.all().select_related('author', 'author__profile')
.prefetch_related('categories', 'tags')
.only('id', 'title', 'slug', 'status', 'created_at',
      'author__id', 'author__username', 'author__email')

# Use Prefetch object for filtered prefetch
from django.db.models import Prefetch
published_comments = Comment.objects.filter(approved=True)
Article.objects.prefetch_related(
    Prefetch('comments', queryset=published_comments,
            to_attr='approved_comments')
)
```

■ *Install `django-debug-toolbar` in development and check the SQL panel. Every list endpoint should show a small, fixed number of queries regardless of page size.*

## 6.2 Django Cache Framework

### ■ settings.py — Redis cache

```
pip install django-redis

CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
        "OPTIONS": {
```

```

"CLIENT_CLASS": "django_redis.client.DefaultClient",
"SOCKET_CONNECT_TIMEOUT": 5,
"SOCKET_TIMEOUT": 5,
"CONNECTION_POOL_KWARGS": {"max_connections": 50},
},
"KEY_PREFIX": "myapp",
}
}

SESSION_ENGINE = "django.contrib.sessions.backends.cache"
SESSION_CACHE_ALIAS = "default"

```

#### ■ Manual cache in views

```

from django.core.cache import cache

class ArticleViewSet(viewsets.ModelViewSet):
    def list(self, request, *args, **kwargs):
        cache_key = f"article_list_{request.query_params.urlencode()}"
        cached = cache.get(cache_key)
        if cached:
            return Response(cached)

        response = super().list(request, *args, **kwargs)
        cache.set(cache_key, response.data, timeout=60*5) # 5 minutes
        return response

    def perform_create(self, serializer):
        serializer.save(author=self.request.user)
        # Invalidate list cache on write
        cache.delete_pattern("article_list_*")

```

## 6.3 Response Caching with `cache_page`

#### ■ Per-view cache decorator

```

from django.utils.decorators import method_decorator
from django.views.decorators.cache import cache_page
from django.views.decorators.vary import vary_on_headers

# Cache for 15 min, vary by Authorization header (per-user cache)
@method_decorator(cache_page(60 * 15), name='dispatch')
@method_decorator(vary_on_headers("Authorization"), name='dispatch')
class ArticleListView(generics.ListAPIView):
    ...

```

---

## 6.4 Database Indexing

---

### ■ Model indexes for DRF filter fields

```
class Article(models.Model):
    ...
    class Meta:
        indexes = [
            # Single-column – for filtering by status
            models.Index(fields=['status'], name='article_status_idx'),
            # Composite – for filtering status + ordering by date
            models.Index(fields=['status', '-created_at'],
                          name='article_status_date_idx'),
            # Partial index (PostgreSQL) – only published articles
            models.Index(
                fields=['created_at'],
                condition=models.Q(status='published'),
                name='article_pub_date_idx'
            ),
        ]
```



# Stage 7 · Testing & Documentation

Untested APIs are bugs waiting to happen. Undocumented APIs are dead APIs.

## 7.1 Testing with APIClient

■ tests/test\_articles.py

```
from rest_framework.test import APITestCase, APIClient
from rest_framework import status
from django.urls import reverse
from django.contrib.auth import get_user_model
from .models import Article

User = get_user_model()

class ArticleAPITests(APITestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            username='jane', password='pass1234', email='jane@example.com'
        )
        self.other = User.objects.create_user(
            username='bob', password='pass5678', email='bob@example.com'
        )
        self.article = Article.objects.create(
            title='Test Article', slug='test-article',
            body='Body text here.', author=self.user, status='published'
        )
        self.list_url = reverse('article-list')
        self.detail_url = reverse('article-detail', kwargs={'slug': self.article.slug})

    def authenticate(self, user):
        self.client.force_authenticate(user=user)

    def test_list_unauthenticated(self):
        response = self.client.get(self.list_url)
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_create_authenticated(self):
        self.authenticate(self.user)
        data = {'title': 'New Article', 'body': 'Content', 'status': 'draft'}
        response = self.client.post(self.list_url, data, format='json')
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
```

```

self.assertEqual(response.data['title'], 'New Article')

def test_create_unauthenticated_forbidden(self):
    data = {'title': 'New', 'body': 'Content'}
    response = self.client.post(self.list_url, data, format='json')
    self.assertEqual(response.status_code, status.HTTP_401_UNAUTHORIZED)

def test_update_own_article(self):
    self.authenticate(self.user)
    data = {'title': 'Updated Title', 'body': 'Updated body'}
    response = self.client.patch(self.detail_url, data, format='json')
    self.assertEqual(response.status_code, status.HTTP_200_OK)
    self.assertEqual(response.data['title'], 'Updated Title')

def test_update_others_article_forbidden(self):
    self.authenticate(self.other)
    response = self.client.patch(self.detail_url, {'title': 'Hack'}, format='json')
    self.assertEqual(response.status_code, status.HTTP_403_FORBIDDEN)

def test_delete_article(self):
    self.authenticate(self.user)
    response = self.client.delete(self.detail_url)
    self.assertEqual(response.status_code, status.HTTP_204_NO_CONTENT)
    self.assertFalse(Article.objects.filter(slug='test-article').exists())

```

## 7.2 pytest-django + factory\_boy

### ■ factories.py

```

import factory

from factory.django import DjangoModelFactory
from django.contrib.auth import get_user_model
from .models import Article, Category

User = get_user_model()

class UserFactory(DjangoModelFactory):
    class Meta:
        model = User

    username = factory.Sequence(lambda n: f'user{n}')
    email = factory.LazyAttribute(lambda o: f'{o.username}@example.com')
    password = factory.PostGenerationMethodCall('set_password', 'pass1234')
    is_active = True

class CategoryFactory(DjangoModelFactory):
    class Meta:

```

```

model = Category
name = factory.Faker('word')
slug = factory.LazyAttribute(lambda o: o.name.lower())

class ArticleFactory(DjangoModelFactory):
    class Meta:
        model = Article
    title = factory.Faker('sentence', nb_words=5)
    slug = factory.Sequence(lambda n: f'article-{n}')
    body = factory.Faker('paragraphs', nb=3, as_text=True)
    author = factory.SubFactory(UserFactory)
    status = 'published'

    @factory.post_generation
    def categories(self, create, extracted, **kwargs):
        if not create:
            return
        if extracted:
            for cat in extracted:
                self.categories.add(cat)

```

#### ■ tests/test\_articles\_pytest.py

```

import pytest
from rest_framework.test import APIClient
from .factories import UserFactory, ArticleFactory

@pytest.fixture
def api_client():
    return APIClient()

@pytest.fixture
def auth_client(api_client):
    user = UserFactory()
    api_client.force_authenticate(user=user)
    return api_client, user

@pytest.mark.django_db
def test_article_list(api_client):
    ArticleFactory.create_batch(5)
    response = api_client.get('/api/v1/articles/')
    assert response.status_code == 200
    assert len(response.data['results']) == 5

@pytest.mark.django_db
def test_create_article(auth_client):
    client, user = auth_client

```

```
data = {'title': 'My Article', 'body': 'Content', 'status': 'draft'}
response = client.post('/api/v1/articles/', data, format='json')
assert response.status_code == 201
assert response.data['author_name'] == user.get_full_name()
```

## 7.3 API Documentation with drf-spectacular

### ■ settings.py + urls.py

```
pip install drf-spectacular

INSTALLED_APPS += ['drf_spectacular']

REST_FRAMEWORK = {
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
}

SPECTACULAR_SETTINGS = {
    'TITLE': 'My API',
    'DESCRIPTION': 'Production-quality Django REST Framework API',
    'VERSION': '1.0.0',
    'SERVE_INCLUDE_SCHEMA': False,
    'COMPONENT_SPLIT_REQUEST': True,
}

# urls.py
from drf_spectacular.views import (
    SpectacularAPIView, SpectacularSwaggerView, SpectacularRedocView
)

urlpatterns += [
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
    path('api/docs/', SpectacularSwaggerView.as_view(url_name='schema')),
    path('api/redoc/', SpectacularRedocView.as_view(url_name='schema')),
]
```

### ■ Annotating views with @extend\_schema

```
from drf_spectacular.utils import extend_schema, OpenApiParameter, OpenApiExample

class ArticleViewSet(viewsets.ModelViewSet):

    @extend_schema(
        summary="List all published articles",
        parameters=[
            OpenApiParameter('search', str, description='Search in title/body'),
            OpenApiParameter('status', str, enum=['draft', 'published']),
        ],
```

---

```
responses={200: ArticleSerializer(many=True)},
examples=[
    OpenApiExample('Success', value={'count':1,'results':[{'title':'...'}]})
]
)

def list(self, request, *args, **kwargs):
    return super().list(request, *args, **kwargs)
```

# Stage 8 · Async, Tasks & Real-time

Background jobs, WebSockets, and async views for modern applications

## 8.1 Celery + Redis

Celery is the standard task queue for Django. Use it for anything that shouldn't block the HTTP response: emails, image processing, notifications, reports.

### ■ celery.py + tasks.py

```
# myproject/celery.py
import os
from celery import Celery

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'myproject.settings.production')
app = Celery('myproject')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()

# settings.py
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'
CELERY_RESULT_SERIALIZER = 'json'
CELERY_TIMEZONE = 'UTC'
CELERY_TASK_ALWAYS_EAGER = False # set True in tests

# articles/tasks.py
from celery import shared_task
from django.core.mail import send_mail

@shared_task(bind=True, max_retries=3, default_retry_delay=60)
def send_publish_notification(self, article_id):
    try:
        from .models import Article
        article = Article.objects.get(id=article_id)
        send_mail(
            subject=f'New article: {article.title}',
            message=article.body[:200],
            from_email='noreply@example.com',
            recipient_list=[article.author.email],
```

```

)
except Exception as exc:
    raise self.retry(exc=exc)

# Trigger from a view
def perform_create(self, serializer):
    article = serializer.save(author=self.request.user)
    if article.status == 'published':
        send_publish_notification.delay(article.id)

# Start workers
# celery -A myproject worker -l info -c 4

```

## 8.2 Celery Beat — Scheduled Tasks

### ■ Periodic tasks

```

pip install django-celery-beat

INSTALLED_APPS += ['django_celery_beat']

# settings.py
CELERY_BEAT_SCHEDULER = 'django_celery_beat.schedulers:DatabaseScheduler'

# tasks.py
from celery import shared_task

@shared_task
def cleanup_draft_articles():
    from datetime import timedelta
    from django.utils import timezone
    cutoff = timezone.now() - timedelta(days=30)
    deleted, _ = Article.objects.filter(
        status='draft', updated_at__lt=cutoff
    ).delete()
    return f"Deleted {deleted} stale drafts"

@shared_task
def generate_weekly_report():
    # Generate and email a report
    ...

# celery.py – programmatic schedule
from celery.schedules import crontab

app.conf.beat_schedule = {
    'cleanup-drafts-daily': {
        'task': 'articles.tasks.cleanup_draft_articles',

```

```
'schedule': crontab(hour=2, minute=0), # 2am UTC daily
},
'weekly-report': {
'task': 'articles.tasks.generate_weekly_report',
'schedule': crontab(day_of_week='monday', hour=8),
},
}

# celery -A myproject beat -l info
```

## 8.3 Django Channels & WebSockets

### ■ channels setup + consumer

```
pip install channels channels-redis

# settings.py
INSTALLED_APPS += ['channels']
ASGI_APPLICATION = 'myproject.asgi.application'

CHANNEL_LAYERS = {
'default': {
'BACKEND': 'channels_redis.core.RedisChannelLayer',
'CONFIG': {'hosts': [('127.0.0.1', 6379)]},
},
}

# myproject/asgi.py
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from notifications.routing import websocket_urlpatterns

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'myproject.settings.production')

application = ProtocolTypeRouter({
'http': get_asgi_application(),
'websocket': AuthMiddlewareStack(
URLRouter(websocket_urlpatterns)
),
})

# notifications/consumers.py
import json
from channels.generic.websocket import AsyncWebsocketConsumer
```



```

class NotificationConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        user = self.scope['user']
        if not user.is_authenticated:
            await self.close()
            return
        self.group_name = f'notifications_{user.id}'
        await self.channel_layer.group_add(
            self.group_name, self.channel_name
        )
        await self.accept()

    async def disconnect(self, code):
        await self.channel_layer.group_discard(
            self.group_name, self.channel_name
        )

    async def notification_message(self, event):
        await self.send(text_data=json.dumps(event['data']))

    # Send a notification from anywhere:
    from channels.layers import get_channel_layer
    from asgiref.sync import async_to_sync

    def send_notification(user_id, data):
        layer = get_channel_layer()
        group_name = f'notifications_{user_id}'
        async_to_sync(layer.group_send)(
            group_name,
            {'type': 'notification.message', 'data': data}
        )

```

# Stage 9 · Security Hardening

Lock down your API before it goes anywhere near the internet

## 9.1 Throttling (Rate Limiting)

Throttling prevents abuse by limiting how many requests a client can make in a time window. Always throttle unauthenticated users more aggressively.

### ■ settings.py + custom throttle

```
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle',
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '100/day',
        'user': '1000/day',
        'login': '5/min', # custom scope
        'burst': '60/min',
    }
}

# throttles.py – custom throttle
from rest_framework.throttling import ScopedRateThrottle

class LoginRateThrottle(ScopedRateThrottle):
    scope = 'login'

# views.py
class LoginView(APIView):
    throttle_classes = [LoginRateThrottle]
    authentication_classes = []
    permission_classes = []
    ...
```

## 9.2 CORS

### ■ django-cors-headers setup

```
pip install django-cors-headers
```

```

INSTALLED_APPS += ['corsheaders']
MIDDLEWARE.insert(0, 'corsheaders.middleware.CorsMiddleware')

# Development
CORS_ALLOW_ALL_ORIGINS = True # NEVER in production

# Production
CORS_ALLOWED_ORIGINS = [
    'https://app.example.com',
    'https://admin.example.com',
]
CORS_ALLOW_CREDENTIALS = True
CORS_ALLOW_HEADERS = [
    'accept', 'accept-encoding', 'authorization',
    'content-type', 'dnt', 'origin', 'user-agent', 'x-csrftoken',
    'x-requested-with',
]
CORS_ALLOW_METHODS = ['DELETE', 'GET', 'OPTIONS', 'PATCH', 'POST', 'PUT']

```

## 9.3 HTTPS & Security Headers

### ■ settings/production.py — security hardening

```

# HTTPS
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000 # 1 year
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True

# Cookies
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = 'Lax'
CSRF_COOKIE_SECURE = True
CSRF_COOKIE_HTTPONLY = True

# Click-jacking
X_FRAME_OPTIONS = 'DENY'
SECURE_CONTENT_TYPE_NOSNIFF = True
SECURE_BROWSER_XSS_FILTER = True # legacy, still useful

# Content Security Policy (requires django-csp)
CSP_DEFAULT_SRC = ('self',)
CSP_SCRIPT_SRC = ('self', 'https://cdn.example.com')

```

```
CSP_STYLE_SRC = ('self', 'unsafe-inline')
```

## 9.4 Brute-Force Protection with django-axes

### ■ django-axes setup

```
pip install django-axes

INSTALLED_APPS += ['axes']
MIDDLEWARE += ['axes.middleware.AxesMiddleware']

AUTHENTICATION_BACKENDS = [
    'axes.backends.AxesStandaloneBackend',
    'django.contrib.auth.backends.ModelBackend',
]

# Settings
AXES_FAILURE_LIMIT = 5 # lock after 5 failed attempts
AXES_COOLOFF_TIME = 1 # lockout for 1 hour
AXES_LOCKOUT_PARAMETERS = ['username', 'ip_address']
AXES_RESET_ON_SUCCESS = True # reset counter on successful login
AXES_ENABLE_ACCESS_FAILURE_LOG = True
```

# Stage 10 · Deployment & Production Ops

Get it live, keep it running, know when it breaks

## 10.1 Gunicorn + Uvicorn

### ■ gunicorn.conf.py

```
# gunicorn.conf.py (WSGI — no WebSockets)

bind = "0.0.0.0:8000"

workers = 4 # 2 * CPU cores + 1

worker_class = "gthread"

threads = 4

timeout = 30

keepalive = 5

max_requests = 1000

max_requests_jitter = 100 # stagger restarts

accesslog = "-" # stdout

errorlog = "-"

loglevel = "info"

preload_app = True # fork after loading to save memory

# Launch:

# gunicorn myproject.wsgi:application -c gunicorn.conf.py

# --- For ASGI (Django Channels / async) ---

# uvicorn.run.sh

# uvicorn myproject.asgi:application # --host 0.0.0.0 --port 8000 # --workers 4
--loop uvloop --http httptools
```

## 10.2 Docker & Docker Compose

### ■ Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1

ENV PYTHONUNBUFFERED=1

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends build-essential
libpq-dev && rm -rf /var/lib/apt/lists/*
```

```
COPY requirements/production.txt .
RUN pip install --no-cache-dir -r production.txt

COPY . .

RUN python manage.py collectstatic --no-input

EXPOSE 8000

CMD ["gunicorn", "myproject.wsgi:application", "-c", "gunicorn.conf.py"]
```

#### ■ docker-compose.yml

```
version: "3.9"
services:
  db:
    image: postgres:16
    volumes: [postgres_data:/var/lib/postgresql/data]
    env_file: .env
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER"]
      interval: 5s; retries: 5

  redis:
    image: redis:7-alpine
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru

  web:
    build: .
    command: gunicorn myproject.wsgi:application -c gunicorn.conf.py
    volumes: [./:/app, static_volume:/app/staticfiles]
    expose: ["8000"]
    env_file: .env
    depends_on: {db: {condition: service_healthy}}

  celery:
    build: .
    command: celery -A myproject worker -l info -c 4
    env_file: .env
    depends_on: [db, redis]

  celery-beat:
    build: .
    command: celery -A myproject beat -l info
    env_file: .env
    depends_on: [redis]

  nginx:
    image: nginx:alpine
```

```
ports: ["80:80", "443:443"]
volumes:
- ./nginx.conf:/etc/nginx/conf.d/default.conf
- static_volume:/staticfiles:ro
- ./certs:/etc/ssl/certs:ro
depends_on: [web]

volumes: {postgres_data: {}, static_volume: {}}
```

## 10.3 Nginx Configuration

### ■ nginx.conf

```
upstream django {
server web:8000;
}

server {
listen 80;
server_name example.com www.example.com;
return 301 https://$host$request_uri;
}

server {
listen 443 ssl http2;
server_name example.com www.example.com;

ssl_certificate /etc/ssl/certs/fullchain.pem;
ssl_certificate_key /etc/ssl/certs/privkey.pem;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;

# Security headers
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload";
add_header X-Frame-Options DENY;
add_header X-Content-Type-Options nosniff;

client_max_body_size 20M;

location /static/ {
alias /staticfiles/;
expires 30d;
add_header Cache-Control "public, immutable";
}

location /media/ {
alias /mediafiles/;
```

```
expires 7d;
}

location / {
    proxy_pass http://django;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 90;
    proxy_connect_timeout 90;
}
}
```

## 10.4 Logging & Monitoring with Sentry

### ■ settings/production.py — structured logging + Sentry

```
pip install sentry-sdk

import sentry_sdk
from sentry_sdk.integrations.django import DjangoIntegration
from sentry_sdk.integrations.celery import CeleryIntegration
from sentry_sdk.integrations.redis import RedisIntegration

sentry_sdk.init(
    dsn=env('SENTRY_DSN'),
    integrations=[
        DjangoIntegration(transaction_style='url'),
        CeleryIntegration(),
        RedisIntegration(),
    ],
    traces_sample_rate = 0.1, # 10% of transactions for performance
    send_default_pii = False,
    environment = 'production',
    release = env('GIT_SHA', default='unknown'),
)

LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'json': {
            '():': 'pythonjsonlogger.jsonlogger.JsonFormatter',
```



```
'fmt': '%(asctime)s %(name)s %(levelname)s %(message)s',
},
},
'handlers': {
'console': {
'class': 'logging.StreamHandler',
'formatter': 'json',
},
},
'root': {'level': 'INFO', 'handlers': ['console']},
'loggers': {
'django': {'level': 'INFO', 'propagate': True},
'django.request': {'level': 'WARNING', 'propagate': True},
'myapp': {'level': 'DEBUG', 'propagate': True},
},
}
```

## 10.5 CI/CD with GitHub Actions

### ■ .github/workflows/ci.yml

```
name: CI

on: [push, pull_request]

jobs:
test:
runs-on: ubuntu-latest
services:
postgres:
image: postgres:16
env:
POSTGRES_PASSWORD: postgres
POSTGRES_DB: test_db
options: >-
--health-cmd pg_isready
--health-interval 10s
--health-timeout 5s
--health-retries 5
redis:
image: redis:7-alpine
options: --health-cmd "redis-cli ping"

steps:
```

```
- uses: actions/checkout@v4

- uses: actions/setup-python@v5
with: {python-version: '3.12'}

- name: Install deps
run: |
pip install -r requirements/development.txt

- name: Run tests
env:
DATABASE_URL: postgres://postgres:postgres@localhost/test_db
CELERY_TASK_ALWAYS_EAGER: 'True'
run: |
python manage.py migrate
pytest --cov=. --cov-report=xml -v

- name: Upload coverage
uses: codecov/codecov-action@v4

deploy:
needs: test
runs-on: ubuntu-latest
if: github.ref == 'refs/heads/main'
steps:
- uses: actions/checkout@v4
- name: Deploy to production
run: |
ssh user@server 'cd /app && git pull && docker compose up -d --build'
```

## Appendix — Quick Reference

### HTTP Status Codes in DRF

| Code | DRF Constant                   | When to use                     |
|------|--------------------------------|---------------------------------|
| 200  | HTTP_200_OK                    | GET, PUT, PATCH success         |
| 201  | HTTP_201_CREATED               | POST created a resource         |
| 204  | HTTP_204_NO_CONTENT            | DELETE success                  |
| 400  | HTTP_400_BAD_REQUEST           | Validation error                |
| 401  | HTTP_401_UNAUTHORIZED          | Not authenticated               |
| 403  | HTTP_403_FORBIDDEN             | Authenticated but no permission |
| 404  | HTTP_404_NOT_FOUND             | Resource not found              |
| 429  | HTTP_429_TOO_MANY_REQUESTS     | Throttle exceeded               |
| 500  | HTTP_500_INTERNAL_SERVER_ERROR | Unhandled exception             |

### Essential Packages

| Package                         | Purpose                   |
|---------------------------------|---------------------------|
| django-rest-framework           | Core DRF                  |
| django-rest-framework-simplejwt | JWT authentication        |
| django-filter                   | Advanced filtering        |
| drf-spectacular                 | OpenAPI 3 schema & docs   |
| django-cors-headers             | CORS headers              |
| django-redis                    | Redis cache backend       |
| celery                          | Async task queue          |
| django-celery-beat              | Scheduled tasks           |
| channels + channels-redis       | WebSockets                |
| factory_boy                     | Model factories for tests |
| pytest-django                   | pytest integration        |
| sentry-sdk                      | Error tracking            |
| django-axes                     | Brute-force protection    |
| django-environ                  | 12-factor config          |
| gunicorn / uvicorn              | WSGI / ASGI servers       |